UiA University of Agder

Master

Reflect on Discrete Behaviour

Matin Mohammadi, Robert Hanssen, Sander Gnanavel

IKT467-1 25H

University of Agder, 2025
Faculty of Engineering and Science
Department of Engineering and Sciences

Obligatorisk gruppeerklæring

Den enkelte student er selv ansvarlig for å sette seg inn i hva som er lovlige hjelpemidler, retningslinjer for bruk av disse og regler om kildebruk. Erklæringen skal bevisstgjøre studentene på deres ansvar og hvilke konsekvenser fusk kan medføre. Manglende erklæring fritar ikke studentene fra sitt ansvar.

1.	Vi erklærer herved at vår besvarelse er vårt eget arbeid, og at vi ikke har brukt andre kilder eller har mottatt annen hjelp enn det som er nevnt i besvarelsen.	Ja / Nei
2.	**Vi erklærer videre at denne besvarelsen:** • Ikke har vært brukt til annen eksamen ved annen avdeling/universitet/høgskole innenlands eller utenlands. • Ikke refererer til andres arbeid uten at det er oppgitt. • Ikke refererer til eget tidligere arbeid uten at det er oppgitt. • Har alle referansene oppgitt i litteraturlisten. • Ikke er en kopi, duplikat eller avskrift av andres arbeid eller besvarelse.	Ja / Nei
3.	Vi er kjent med at brudd på ovennevnte er å betrakte som fusk og kan medføre annullering av eksamen og utestengelse fra universiteter og høgskoler i Norge, jf. Universitets- og høgskoleloven §§4-7 og 4-8 og Forskrift om eksamen §§ 31.	Ja / Nei
4.	Vi er kjent med at alle innleverte oppgaver kan bli plagiatkontrollert.	Ja / Nei
5.	Vi er kjent med at Universitetet i Agder vil behandle alle saker hvor det forligger mistanke om fusk etter høgskolens retningslinjer for behandling av saker om fusk.	Ja / Nei
6.	Vi har satt oss inn i regler og retningslinjer i bruk av kilder og referanser på biblioteket sine nettsider.	Ja / Nei
7.	Vi har i flertall blitt enige om at innsatsen innad i gruppen er merkbart forskjellig og ønsker dermed å vurderes individuelt. Ordinært vurderes alle deltakere i prosjektet samlet.	Ja / Nei

Publiseringsavtale

Fullmakt til elektronisk publisering av oppgaven Forfatter(ne) har opphavsrett til oppgaven. Det betyr blant annet enerett til å gjøre verket tilgjengelig for allmennheten (Åndsverkloven. §2).
Oppgaver som er unntatt offentlighet eller taushetsbelagt/konfidensiell vil ikke bli publisert.

Vi gir herved Universitetet i Agder en vederlagsfri rett til å gjøre oppgaven tilgjengelig for elektronisk publisering:	Ja / Nei
Er oppgaven båndlagt (konfidensiell)?	Ja / Nei
Er oppgaven unntatt offentlighet?	Ja / Nei

Contents

Chapter 1

Model time

We use a discrete global clock with fixed step size Δt. The model advances in fixed increments called time steps of length Δt inside the simulation loop. The set of model time points is $\{t_0, t_0 + \Delta t, t_0 + 2\Delta t, \ldots, t_{\text{End}}\}$. The first time point is the start of the run, and after that the clock advances by one step at a time until the total simulation duration `sim_dur_s` is reached. A deterministic seed ensures that the same inputs always produce the same trajectory set and we have implemented a warm up of fixed duration. The data recording starts at $t \geq t_{\text{warmup}}$, so that only post–warmup snapshots are recorded.

At each time point t_k, the engine performs five ordered stages to avoid race conditions:

1. **Perception.** Each vehicle reads the signals it needs from the previous state, such as its own speed, the headway gap to the leader, and relative speed.
2. **Decision.** The driver logic converts those perceived values into a target acceleration. `HumanDriver` uses `aggression`, `gap_s`, and `reaction_time_s`. `AutonomousDriver` uses `communication_delay_s`, `gap_s`, and `reaction_time_s`.
3. **Actuation.** The requested acceleration is limited by comfort and vehicle constraints, then clamped to the legal range for the road segment.
4. **Integration.** The model updates each vehicle's speed and position using forward Euler integration[2, 1] over $[t_k, t_{k+1}]$:

$$v_{k+1} = \big(v_k + a_k\,\Delta t,\, 0,\, v_{\max}\big), \qquad x_{k+1} = \big(x_k + v_{k+1}\,\Delta t,\, x_{\min},\, x_{\max}\big) \tag{1.1}$$

   This ensures speed cannot go below zero or exceed the limit, and position remains within road bounds.
5. **Logging.** The engine records the new motion state (`Kinematics`) for each vehicle and any aggregates required for KPI analysis.

Between time points, the state of the model remains constant, no changes occur. Visualisations may interpolate for smooth plots, but all true updates happen only at discrete ticks. If two events would occur at the same time, the engine resolves them deterministically using the update order and the vehicle identifier to ensure repeatable results.

Chapter 2

Attribute values

2.1 How attributes change.

At each time point the driver logic proposes an acceleration based on perception. The engine limits that value to safe and legal bounds. Then speed is updated using the limited acceleration and the time step. Then position is updated using the new speed. If a calculated value falls outside its legal range it is clipped to the nearest legal bound and a counter records that it happened. If too many such corrections occur the run is flagged.

2.2 Relation to precision and accuracy.

Precision is how many decimals we keep when storing values. We choose precision to balance smooth plots and stable metrics against file size. Positions and vehicle length use one tenth of a meter. Speeds and accelerations use one hundredth in their units. Road length uses one meter. Speed limit uses one tenth in its unit. Counts and identifiers are integers. Accuracy is how close the numbers are to the intended physical values. The main source of error is the time step. A shorter time step improves accuracy but increases runtime and file size. We pick a time step so that updates in speed during one step are small compared to typical speeds. This keeps the key metrics such as throughput and travel time within acceptable error.

2.3 Acceptable and Unacceptable Values

2.3.1 Identifiers and flags

`vehicle_id` and `road_id` must be positive integers and unique within a simulation run. Unacceptable values include negative numbers, zero, missing values, or duplicates. The `autonomous` flag must be strictly `true` or `false`; missing values or any other type are invalid.

2.3.2 Road

The `segment_length` (meters) must be greater than zero; zero or negative values are invalid. The `speed_limit` (m/s) must also be greater than zero; zero or negative values are unacceptable. The `capacity_per_km` must be a non-negative integer; negative or non-integer values are not allowed.

2.3.3 Vehicle.

The `vehicle_length` (meters) must be greater than zero; zero or negative values are unacceptable. The model uses only a single value for `acceleration`. The `max_acceleration` (m/s^2) must be greater than zero and `max_deceleration` (m/s^2) must be less than zero.

2.3.4 Kinematics.

The `position` (meters) must remain within road bounds, i.e., between zero and the segment length. Unacceptable values are those below zero or above the segment length. The `speed` (m/s) must remain within the interval [0, speed_limit + 15%]. Unacceptable values are below zero or exceed the limit. The `acceleration` (m/s^2) must remain within [max_deceleration, max_acceleration]. Values outside this range are invalid. When this value is negative, it represents deceleration. A value of zero or a positive number is valid for acceleration, while a negative number defines the braking capability.

2.3.5 Driver.

The `reaction_time_s` must be greater than zero; zero or negative values are unacceptable. The `aggression` attribute must be a string label drawn from a defined set; missing or unrecognized labels are invalid. The `communication_delay_s` must be zero or greater; negative values are not allowed.

2.3.6 Relation to precision and accuracy.

Precision refers to the number of decimal places used for storage. Limiting precision affects rounding only and does not alter the set of acceptable values. *Accuracy* describes how closely computed quantities match their intended physical values. The main source of accuracy error is the discrete time step, not numerical precision. Acceptable ranges are enforced at every time point. If a computed value exceeds its legal range, it is clipped to the nearest valid bound and the correction is recorded. If the number or magnitude of corrections surpasses a predefined threshold, the simulation run is flagged for review.

Chapter 3

Agent behaviours

The model consists of a single road environment and `Vehicle agents`, each controlled by a `Driver policy`. The simulation runs in discrete time steps where at each time point, vehicles perceive their current state, and decide on a target acceleration. The kinematics of the vehicle is updated based on this decision using Euler integration.

3.1 Driver policies

We use two different driver types. Type one is the `HumanDriver class`, which models how people drive, with an additional parameter `aggression`. It tries to achieve the desired speed when there is an open road, and keeps a safe distance when following another car. The aggressive drivers accelerate harder and leave smaller gaps, while calm drivers do the opposite. The `AutonomousDriver class` is the second type, and it represents self-driving cars. These use tighter following distances and more precise control, reflecting how autonomous vehicles can communicate and coordinate better than humans. They have smaller gap requirements and faster reaction times than human drivers. Both types calculate a desired acceleration based on how far away the car in front is, their current speed, and how fast they are closing the gap.

3.2 Perception and ordering

At the start of each time step, we sort all vehicles by their position on the road. This sorting is important because each vehicle needs to know who's directly in front of them. We handle the edges of the road depending on which boundary mode we are using. With `periodic boundaries`, the road wraps around like a ring, and with `open boundaries`, cars that reach the end of the road just exit the simulation.

3.3 Scenario specific behaviour

Researchers using the model may run preconfigured simulated scenarios to study specific traffic phenomena. For example, we attempt to replicate the `Phantom Jam` phenomenon. For this scenario, the model will, at a set time, suddenly slow down all cars in a specific zone. The idea is that even though no actual obstacle exists, this creates a shockwave that travels backward through traffic, causing stop-and-go patterns. The scenario rules apply after the driver calculates their desired acceleration but before we update the vehicle.

3.4 Integration and constraints

The kinematic state of each vehicle is updated using the explicit Euler integration method (equation: 1.1) and a clip function. v_k and x_k are the speed and position at the current time step t_k, a_k is the final, constrained acceleration applied during the interval $[t_k, t_{k+1}]$, and Δt is the time step duration. The `clip` function ensures that the speed remains positive and does not exceed an upper limit (115% of the speed limit).

Chapter 4

State diagrams

4.1 Vehicle

Figure 4.1 describes what kind of movement pattern a vehicle is in at any moment. In `FreeFlow`, the vehicle mainly cares about reaching the speed limit. There are no other cars in sight to worry about, so it just accelerates comfortably toward its target speed. In `CarFollowing`, the vehicle is following and is focused on the car ahead. It will adjusts its speed to maintain a safe gap, balancing the need to keep moving with the need to stay safe. The vehicle might speed up or slow down slightly to find the right spacing for its current speed.

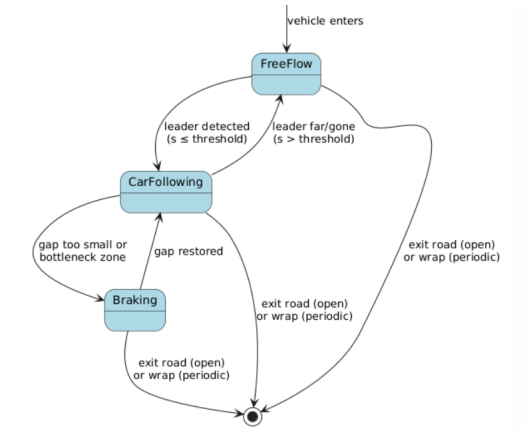

Figure 4.1: Vehicle Deterministic Finite State Machine

4.2 Driver

Figure 4.2 depicts the decision process of the driver. In `Cruise`, the driver is mainly thinking about the target speed (speed limit). Following distance does not matter much because either there is no leader, or they are too far away. In `Follow`, the interaction with the leader matters. The driver is actively managing the gap, making small adjustments to acceleration to keep the right spacing. In `HardBrake`, the driver is applying close to maximum braking. This is the emergency response state, applied when something requires urgent slowing down. This may be a closing gap or an external constraint like a bottleneck. The required deceleration approaches or may exceed the comfort limit.

The transition from `Cruise` to `Follow` occurs when:
$S_{physical\_gap} \leq S_{\text{following\_treshold}}(v)$.

The reverse happens when:
$S_{physical\_gap} > S_{\text{following\_treshold}}(v)$ or the leader is lost.

The transition to `HardBrake` is triggered when:
$a_{\text{desired\_acc}} < -b_{\text{treshold}}$, where b_{treshold} is a hard braking threshold.

The system exits `HardBrake` when:
$a_{\text{desired\_acc}} \geq -b_{\text{treshold}}$.

The threshold values differ between `HumanDriver` and `AutonomousDriver` agents. For example, aggressive human drivers have larger a and smaller gap parameters compared to calm drivers.

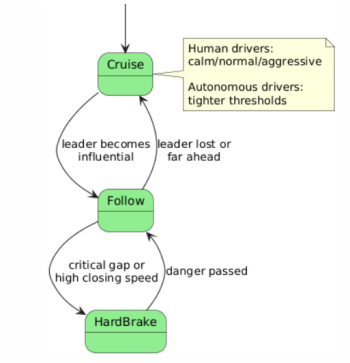

Figure 4.2: Driver Deterministic Finite State Machine

4.3 Scenario

Figure 4.3 visualizes the overall simulation flow and when special events happen. We start in `Warmup`, where the simulation runs but we don't record data. This lets the traffic settle into a natural patterns rather than analyzing the artificial initial conditions. The system remains in this state until the simulation time reaches the predefined warmup duration t_{warmup}, at which point it transitions to `Active mode`. For the `phantom jam` scenario, we briefly enter a `PhantomJamTrigger` state at time $t = t_{\text{trigger}}$, apply changes to vehicles in the trigger zone, and immediately return to `Active mode` in the next time step.

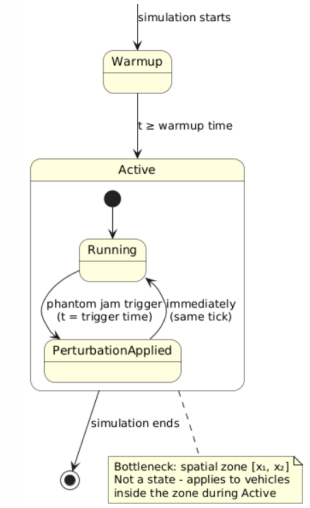

Figure 4.3: Scenario Deterministic Finite State Machine

Chapter 5

Randomness

5.1 Randomness in the Model

5.1.1 Where randomness enters

Randomness enters the model in several ways. When vehicles are created, the driver type can be chosen from a probability set controlled by p_autonomous. Initial positions and speeds can be drawn from distributions so the road does not begin in a perfect grid. Human driver properties such as reaction time, desired gap, and aggression can vary from one vehicle to another. For autonomous drivers, the communication delay can include a small random variation. When the road is open, the arrival of vehicles at the boundary can be random. Lane change decisions may also use a small random term to break ties when two vehicles would otherwise act at the same moment. All random choices use a fixed seed so that a run can be repeated exactly. Changing the seed gives a different but equally valid version of the same experiment.

5.1.2 Background in the real world

In real traffic there is always natural variation. Human drivers do not react in exactly the same way each time, and their perception and timing differ. Vehicle spacing and headways are not fixed and show a range of values. Small disturbances and uneven demand cause waves in the traffic flow. Even control and communication systems in autonomous vehicles have small timing noise. A realistic model must include this variation so that stop-and-go waves and phantom jams can appear on their own, without any fixed obstacle.

5.1.3 What this means for correctness

Correctness in this model is not about matching one run exactly. Each run is one possible outcome for the same inputs. We check the model by looking at the results from many runs. We report the average and the spread for key measures such as throughput, mean travel time, total delay, and density. These results are compared to a reference tool and to known patterns from real traffic. If the same seed and inputs are used, the run must give exactly the same results, showing that the model is consistent. If different seeds are used, the results will vary within a reasonable range, showing that the model behaves in a stable way.

5.1.4 Controls we apply

To control randomness, we fix a seed for all figures in the report so they can be reproduced. We also test several seeds to show that the results are stable. Randomness is never used in safety or rule checks, and all values are limited to legal ranges before they are written to the log. The seed and all input settings are stored together with the outputs. The random

distributions used are simple and described clearly. The time step is chosen so that numerical error stays smaller than the variation caused by randomness.

Bibliography

[1] R. C. Cooper. *Get with the oscillations — Euler and Euler–Cromer*. https://cooperrc.github.io/computational-mechanics/module_03/03_Get_Oscillations.html. Computational Mechanics, CC-BY 4.0. 2020. (Visited on 10/30/2025).

[2] T. Kuan, V. Connelly, A. Cowen, et al. *The Euler Method*. https://pythonnumericalmethods.studentorg.berkeley.edu/notebooks/chapter22.03-The-Euler-Method.html. Python Programming and Numerical Methods: A Guide for Engineers and Scientists. 2019. (Visited on 10/30/2025).